

Project 4 Report

Student / Team Information

- Bryan Edelson - 124040016
- Geoffrey Mikhael - 124040051

Chosen student ID for trace generation: 124040016

Implementation Summary

Task 1:

Parsing the memory address into tag and set index:

- `get_index()` in `memory_hierarchy.cpp`
- `get_tag()` in `memory_hierarchy.cpp`
- `reconstruct_addr()` in `memory_hierarchy.cpp`

The hit/miss logic and statistics + LRU policy:

- Hit detection logic in `CacheLevel::access()`
- Miss handling (victim selection, write-back, fetch, install) in `CacheLevel::access()` and its helpers
- `LRU::onHit()` in `repl_policy.cpp`
- `LRU::onMiss()` in `repl_policy.cpp`
- `LRU::getVictim()` in `repl_policy.cpp`

Task 2:

Connecting L2 (a new `CacheLevel` object) to L1:

- Create L2 `CacheLevel` object in `main.cpp`
- Connect L1 `next_level` pointer to L2 in `main.cpp`
- Set L2 `next_level` pointer to `MainMemory` in `main.cpp`
- Statistics of L2 are automatically there because it is still a `CacheLevel` object, which is already implemented in Task 1

Task 3:

Implementing advanced policies:

- `SRRIP::onHit()` in `repl_policy.cpp`
- `SRRIP::onMiss()` in `repl_policy.cpp`
- `SRRIP::getVictim()` in `repl_policy.cpp`
- `BIP::onHit()` in `repl_policy.cpp`
- `BIP::onMiss()` in `repl_policy.cpp`

- `BIP::getVictim()` in `repl_policy.cpp`

Implementing prefetchers:

- `NextLinePrefetcher::calculatePrefetch()` in `prefetcher.cpp`
- Implement `StridePrefetcher::calculatePrefetch()` in `prefetcher.cpp`
- `install_prefetch()` in `memory_hierarchy.cpp`
- Call prefetcher from `CacheLevel::access()` in `memory_hierarchy.cpp`. Implemented both on hit and miss.

Custom policies and prefetchers:

- `ReverseSRRIP` policy. We don't know what to call it, but it behaves the other way around as normal SRRIP. Prioritize cache line that was hit to be evicted first.
- `RegionPrefetcher` prefetcher. Prefetches a region of radius 64 blocks around a memory block.

Address Mapping Explanation

How was block offset, set index, and tag computed?

- Block offset refers to the bottom bits that should be ignored for the entire address to be a multiple of `block_size` bytes. The number of such bits are $\log_2(\text{block_size})$ from the LSB. Simply ignore these bits by performing a right shift.
- Set index is how a memory block is mapped to the cache set. There will be $\frac{\text{cache_size}}{(\text{block_size} * \text{associativity})}$ (also denoted by `number_of_sets`) sets in the cache. So, $\log_2(\text{number_of_sets})$ bits from the memory address needs to be reserved as the mapping index to cover all possible indices (from 0 to `number_of_sets - 1`). These bits are taken from the first $\log_2(\text{number_of_sets})$ bits from the memory address after dropping the block offset bits.
- Tag is simply just the remaining bits after shifting the memory address by `set_index_bits + block_offset_bits`, that is the upper bits of the memory address.

How do those values above change when cache geometry change?

- Block offset is the offset in a block. So, the range it needs is just $\log_2(\text{block_size})$. Therefore, it changes depending on `block_size`.
- Set index depends on `cache_size`, `block_size`, and `associativity`. Keeping others fixed, as `cache_size` increases, set index bits size increases; as `block_size` or/and `associativity` increases, set index bits size decrease instead.
- Tag bits size decreases as block offset bits or/and set index bits size increase.
- Keep in mind that all the cache parameters here need to be chosen in such a way so that the bits size of them make sense (e.g., no negative size, no size greater than 64 since we only have 64 bits at most, and so on).
- The parameters also need to be powers of two so that their quotient is a power of two

Task 1 Testing

First testing via `make task1` $\Rightarrow$ matches

Other testing via `./cache_sim trace_sanity.txt cache_size associativity block_size L1_latency MEM_latency` :

- `./cache_sim trace_sanity.txt 1 4 256 1 100` , fully-associative
- `./cache_sim trace_sanity.txt 4 4 1024 1 100` , fully-associative but bigger cache size
- `./cache_sim trace_sanity.txt 1 4 512 1 100` , parameters result in unreasonable cache geometry (only 2 sets but 4 associativity) $\Rightarrow$ error, so correct
- `./cache_sim trace_sanity.txt 16 32 64 1 100` , 32-ways associative of the base test. Resulted in 42.07 cycles AMAT (faster), which makes sense because a quick look at `trace_sanity.txt` , it shows some memory block accesses share the same set index bits, so low associativity means it's more likely to get a miss because that particular set is already full.
- `./cache_sim trace_sanity.txt 1048576 1 1 1 100` , 1073741824 number of sets. The program should take very long (which it did) because linear allocation simply does not support that amount of sets.
- `./cache_sim trace_sanity.txt 1048576 1 1024 1 100` , 1048576 number of sets (20 bits are responsible for the index now). It resulted in 36.71 cycles AMAT (faster), which makes sense because we have more sets to utilize now.
- `./cache_sim trace_sanity.txt 1048576 1 1024 2 100` , just changing the L1 cache latency (double now). Result is 37.71 cycles AMAT with total cycles of 2112 (from 2056), in which 2000 were from the main memory accesses (200 accesses in total with 100 cycles latency), so the statistic is correct.
- `./cache_sim trace_sanity.txt 1048576 1 1024 1 500` , just changing the cache and memory latency. Total cycle is now 10056 (from 2056), so also correct.

Overall, from the tests we did, it seems it's correct (even though we don't know rigorously).

Task 2 Hierarchy Explanation

How does L1, L2, and memory interact?

- We always start searching for data in L1
- L1 either returns if present, or consults L2
- L2 responds by returning data if present, or consults MainMemory
- MainMemory definitely has the data and will return it to L2
- L2 will pass it to L1, and L1 pass it to the CPU to be processed
- Each time we consult next level in hierarchy, data latency increases
- On a miss, data from the next line will be copied to the current line. For example on a full miss (have to access data from MainMemory), L2 copies the data from main memory, and L1 copies from L1, creating a new cache line inside them respectively.
- Lastly, since write back mechanism is used, when a dirty line is evicted, it will have to write its content to the next level (this is done inside `CacheLevel::write_back_victim()` helper). A line in a cache

is dirty if its content was written on during runtime. Also, set index and tag uniquely define a memory address, so if content of a line was changed, it corresponds uniquely to a memory address content.

What changed after adding L2?

- As seen in the difference of `make task1` and `make task2` output.
- The total accesses of MainMemory was significantly reduced because L2 helped to serve the misses from L1.
- The total AMAT also dropped because L2 can accomodate more data and still be faster than Main Memory to provide the data to L1.
- Without L2, we would have to pay MainMemory data latency for every L1 miss
- With L2, we have more chance of finding the data in a relatively faster L2 cache before consulting Main Memory and paying the long latency
- If a data happened to be evicted in L1, L2 will most likely have this data stored still (because bigger in size, so it can accommodate). So in future accesses, misses in L1 only need to consult L2 instead of MainMemory.

Task 3 Design Choices

Which replacement policies and prefetchers were implemented?

- LRU, SRRIP, BIP policies.
- NextLine and Stride prefetchers.
- Additionally, we implemented ReverseSRRIP policy and RegionPrefetcher.

We were able to still get under the `BEST_AMAT` without designing our own policies or prefetchers (had to use associativity higher than 8). So, the custom design was mostly just to further lower the AMAT.

Note that in our design, we do not account for overflows of the memory address representation, etc., since this is just a simulation. We just assume the memory address is small enough to not overflow.

Trace Analysis

The student ID used is: 124040016.

The patterns we observed and how it influenced our design:

1. Poor Temporal Locality (minimal loops/repeated accesses)
 - Modified the python script to count the number of zero block deltas.
 - There was only around 100 *zero block deltas* in more than 8000 accesses.
 - Hence, no need to worry too much about polluting cache because it's unlikely we will need old data (this also explains why ReverseSRRIP made our AMAT lower, which prioritizes hit line to be evicted first).
 - That is why we prefetch aggressively for StridePrefetcher (threshold is 1, i.e., as soon as a new stride is detected, always start prefetching), and sample a relatively huge region for RegionPrefetcher (not so large that it becomes detrimental because of potential cache pollution).

- Evidence:

Most frequently accessed blocks

```
4199213: 60 (0.71%)
4199277: 60 (0.71%)
4199341: 60 (0.71%)
4199405: 60 (0.71%)
4199469: 60 (0.71%)
4199533: 60 (0.71%)
4199597: 60 (0.71%)
4199661: 60 (0.71%)
```

- The analysis also shows *most frequent access percentage* still less than 1%
- Sort of confirming the poor temporal locality of the trace (not that much repeated accesses)

2. Stride Dominated

Top block strides (in cache blocks)

```
7: 4052 (48.09%)
1: 2262 (26.85%)
64: 1434 (17.02%)
```

- These 3 strides make up most of the trace, other strides less than 1% each
- This is why L1 StridePrefetcher prefetches the next 2 blocks aggressively so that we can leverage the fast latency of L1, instead of equipping L2 with StridePrefetcher.
- If both L1 and L2 use StridePrefetcher, then it's slower (tested), because L2 does redundant work.

3. Multiple Blocks in One Set

Per-window summary (window size = 256 accesses)

```
[ 2304, 2560) reads= 256 writes= 0 unique_blocks= 23 unique_sets=
2
[ 2560, 2816) reads= 256 writes= 0 unique_blocks= 14 unique_sets=
1
[ 3840, 4096) reads= 243 writes= 13 unique_blocks= 20 unique_sets=
1
[ 4096, 4352) reads= 238 writes= 18 unique_blocks= 160 unique_sets=
64
[ 4352, 4608) reads= 249 writes= 7 unique_blocks= 249 unique_sets=
64
```

- There are some trace window with 20 unique blocks into 1 unique set. This means we need `ASSOC > 20` to accomodate and prevent cache trashing, otherwise L1 misses will increase if we keep throwing out useful data.
- We chose the `ASSOC` of 32 and found any higher would either have no impact or worsen the AMAT. Example (our best is 1.26 with `ASSOC = 32`):

```
Constructed L2: 128KB, 128-way, 4cyc, [LRU + Region]
Constructed L1: 32KB, 128-way, 1cyc, [ReverseSRRIP + Stride]
```

```
=== Starting Simulation ===
```

```
=== Simulation Results ===
```

```
[L1] Hit Rate: 98.50% (Access: 8427, Miss: 126, WB: 161)
      Prefetches Issued: 6248
[L2] Hit Rate: 88.05% (Access: 6535, Miss: 781, WB: 1)
      Prefetches Issued: 114442
[Main Memory] Total Accesses: 115224
```

```
Metrics:
```

```
Total Instructions: 8427
Total Cycles:      12031
AMAT:              1.43 cycles
```

- In this case, high associativity impacts our L2, not L1. One possible explanation from us is that because high associativity means less sets. Although there will be more ways per sets, we utilize L2 a lot for region prefetching. Some useful data from L2 might be evicted too early. In contrast, lower associativity means more sets, so the data in L2 is more distributed.

```
Constructed L2: 128KB, 128-way, 4cyc, [LRU + NextLine]
Constructed L1: 32KB, 128-way, 1cyc, [ReverseSRRIP + Stride]
```

```
=== Starting Simulation ===
```

```
=== Simulation Results ===
```

```
[L1] Hit Rate: 98.50% (Access: 8427, Miss: 126, WB: 161)
      Prefetches Issued: 6248
[L2] Hit Rate: 96.14% (Access: 6535, Miss: 252, WB: 1)
      Prefetches Issued: 1624
[Main Memory] Total Accesses: 1877
```

```
Metrics:
```

```
Total Instructions: 8427
Total Cycles:      11631
AMAT:              1.38 cycles
```

- Here, we change L2 to use `NextLine` prefetcher, improving miss rate, suggesting that our `RegionPrefetcher` prefetcher probably pollutes the small set number we have due to increased ASSOC which causes trashing and eviction of useful cache blocks.

```
Most frequently used L1 sets
```

```
2: 1877 (22.27%)
1: 3302 (39.18%)
3: 1626 (19.30%)
0: 1622 (19.25%)
```

Per-window summary (window size = 256 accesses)

[0,	256)	reads= 225	writes= 31	unique_blocks= 32	unique_sets= 4
[256,	512)	reads= 245	writes= 11	unique_blocks= 193	unique_sets= 4
[512,	768)	reads= 240	writes= 16	unique_blocks= 256	unique_sets= 4
[768,	1024)	reads= 243	writes= 13	unique_blocks= 256	unique_sets= 4
[1024,	1280)	reads= 256	writes= 0	unique_blocks= 256	unique_sets= 4
[1280,	1536)	reads= 256	writes= 0	unique_blocks= 256	unique_sets= 4
[1536,	1792)	reads= 256	writes= 0	unique_blocks= 256	unique_sets= 4
[1792,	2048)	reads= 256	writes= 0	unique_blocks= 256	unique_sets= 4
[2048,	2304)	reads= 256	writes= 0	unique_blocks= 233	unique_sets= 4
[2304,	2560)	reads= 256	writes= 0	unique_blocks= 23	unique_sets= 2
[2560,	2816)	reads= 256	writes= 0	unique_blocks= 14	unique_sets= 1
[2816,	3072)	reads= 252	writes= 4	unique_blocks= 34	unique_sets= 1
[3072,	3328)	reads= 244	writes= 12	unique_blocks= 20	unique_sets= 1
[3328,	3584)	reads= 243	writes= 13	unique_blocks= 20	unique_sets= 1
[3584,	3840)	reads= 243	writes= 13	unique_blocks= 20	unique_sets= 1
[3840,	4096)	reads= 243	writes= 13	unique_blocks= 20	unique_sets= 1
[4096,	4352)	reads= 238	writes= 18	unique_blocks= 160	unique_sets= 4
[4352,	4608)	reads= 249	writes= 7	unique_blocks= 249	unique_sets= 4
[4608,	4864)	reads= 248	writes= 8	unique_blocks= 248	unique_sets= 4
[4864,	5120)	reads= 248	writes= 8	unique_blocks= 248	unique_sets= 4
[5120,	5376)	reads= 248	writes= 8	unique_blocks= 248	unique_sets= 4
[5376,	5632)	reads= 249	writes= 7	unique_blocks= 249	unique_sets= 4
[5632,	5888)	reads= 248	writes= 8	unique_blocks= 249	unique_sets= 4
[5888,	6144)	reads= 248	writes= 8	unique_blocks= 248	unique_sets= 4
[6144,	6400)	reads= 248	writes= 8	unique_blocks= 248	unique_sets= 4
[6400,	6656)	reads= 248	writes= 8	unique_blocks= 248	unique_sets= 4
[6656,	6912)	reads= 249	writes= 7	unique_blocks= 249	unique_sets= 4
[6912,	7168)	reads= 248	writes= 8	unique_blocks= 248	unique_sets= 4
[7168,	7424)	reads= 248	writes= 8	unique_blocks= 248	unique_sets= 4
[7424,	7680)	reads= 248	writes= 8	unique_blocks= 248	unique_sets= 4
[7680,	7936)	reads= 249	writes= 7	unique_blocks= 249	unique_sets= 4
[7936,	8192)	reads= 248	writes= 8	unique_blocks= 249	unique_sets= 4
[8192,	8427)	reads= 228	writes= 7	unique_blocks= 228	unique_sets= 4

- The trace analyzer for the updated ASSOC=128 shows that we have so much unique block demands for many windows but only have limited sets, combined with us prefetching 128 blocks on each access, might pollute the sets.

4. Occasional Access of Data Outside Stride Pattern

- By the time it accesses a data with irregular pattern (occasionally), that data would probably not be inside L1 because L1 continuously performs stride prefetching, so some still useful irregular data might be evicted.
- We already hypothesize that StridePrefetcher would be bad because of redundancy with L1, so we started with NextLinePrefetcher for L2. But then because of this pattern, we equip L2 with RegionPrefetcher (custom-made prefetcher).
- When L1 consults L2, it will cause L2 to prefetch an area around the data requested. This works because L1 would most likely request data in regular strides and L2 would eventually contain the irregular data between the stride lengths. So if L1 ever needs to access those irregular pattern data, L2 has it for L1.

- Prefetching in bulk also doesn't hurt latency because it occurs asynchronously under the assignment simplification assumption, so we can leverage this RegionPrefetcher while being careful not to pollute L2 too much (systematic testing gives 128 blocks as optimal, i.e., ± 64 block radius)
- Leveraging this gives us near perfect Hit Rate for L2:

```

=== Simulation Results ===
[L1] Hit Rate: 98.50% (Access: 8427, Miss: 126, WB: 161)
    Prefetches Issued: 6248
[L2] Hit Rate: 99.43% (Access: 6535, Miss: 37, WB: 1)
    Prefetches Issued: 75757
[Main Memory] Total Accesses: 75795

```

Experimental Results

The cache size is fixed at 32KB, L1 latency at 1 cycle (so L2 latency is 4 cycles), and MEM latency at 100 cycles. For simplicity, the table presented here only shows the AMAT comparison, and only has entries that were meaningful for us:

ASSOC	BLOCK	L1_POL	L1_PREF	L2_POL	L2_PREF	AMAT	NOTE
32	64	LRU	Stride	LRU	NextLine	1.40	No custom
32	64	SRRIP/BIP	Stride	LRU	NextLine	1.41	
32	64	SRRIP/BIP	Stride	SRRIP/BIP	NextLine	1.59	
32	64	ReverseSRRIP	Stride	LRU	NextLine	1.38	ReverseSRRIP good
32	64	ReverseSRRIP	Stride	LRU	Stride	1.68	Confirming not do double Stride
32	64	ReverseSRRIP	Stride	LRU	Region	1.26	Best
32	32	ReverseSRRIP	Stride	LRU	Region	1.36	Decreasing block size
32	128	ReverseSRRIP	Stride	LRU	Region	8.00	Increasing block size
16	32	ReverseSRRIP	Stride	LRU	Region	1.49	Decreasing associativity
64	32	ReverseSRRIP	Stride	LRU	Region	1.39	Increasing associativity

The table above shows how we tried to find the best parameters value.

- For ASSOC, we went with our hypothesized baseline of 32, then tried to lower/heighten the value, but the experimental results showed that 32 is best.

- For the BLOCK_SIZE, we actually settled with 64 first because it was the default value, which turned out to be the best performing.
- For policy, L1 works best with SRRIP/BIP-like policy, whereas L2 policy doesn't change the AMAT at all. This aligns with our hypothesis.
- L1 is mainly for the strided accesses (and they are of poor temporal locality), and should evict lines based on hits/misses (so we also additionally implemented ReverseSRRIP to evict hit lines with highest priority).
- L2's role is to serve the misses from L1 for the irregular pattern accesses. We found that LRU works best because it prioritizes least recently used data in L2 to be evicted. This could be because L1 is already serving the demand for that data so it never consults L2 for that specific data. What needs to be retained in L2 are those things not inside L1, other data that are not being used are perfect candidates to be evicted, hence LRU is good.
- Because of our hypothesis on how to utilize L2, naturally using NextLine for L2 was our initial choice. But then RegionPrefetcher was implemented to fetch a larger region, and it lowered the AMAT. Initially, we tried increasing the *prefetch_count* of NextLine prefetcher to get the next multiple blocks and it worked really well, so we made RegionPrefetcher

Best Configuration and Discussion

Our best configuration is:

```
TASK3_ASSOC ?= 32
TASK3_BLOCK ?= 64
TASK3_L1_POLICY ?= ReverseSRRIP
TASK3_L1_PREFETCH ?= Stride
TASK3_L2_POLICY ?= LRU
TASK3_L2_PREFETCH ?= Region
```

with the following results:

```
Constructed L2: 128KB, 32-way, 4cyc, [LRU + Region]
Constructed L1: 32KB, 32-way, 1cyc, [ReverseSRRIP + Stride]

=== Starting Simulation ===
=== Simulation Results ===
  [L1] Hit Rate: 98.50% (Access: 8427, Miss: 126, WB: 161)
        Prefetches Issued: 6248
  [L2] Hit Rate: 99.43% (Access: 6535, Miss: 37, WB: 1)
        Prefetches Issued: 75757
  [Main Memory] Total Accesses: 75795

Metrics:
  Total Instructions: 8427
  Total Cycles:      10631
  AMAT:              1.26 cycles
```

Why does it perform well? Well, it was already explained in the trace analysis section. But in summary:

- Our trace has a lot of strided accesses. We use L1 to take care of this using StridePrefetcher.
- These strided accesses are mostly forward, so old data is not really useful anymore. That's why ReverseSRRIP policy works well for our trace.
- There might be some irregular data patterns in between the strides. In the case that these data will be required in future accesses, we utilize L2 that has higher size to take care of this. This is also why L2 has the Region prefetcher to fetch more of these irregular data. Keep in mind that we limit the size of the region to prevent severe cache pollution from happening.

Now, because the nature of our prefetcher design (fetching a large region), there may be cache pollution happening. This explains why high associativity (fewer number of sets) doesn't work as well, because there might be too many evicted useful lines (cache pollution)

How/where it will fail?

- Probably when the trace is not dominated by strided accesses
- When a trace has high temporal locality. Because our design exploits low temporal locality to evict old data as soon as possible, so if the trace was of high temporal locality, our design would most likely have poor performance.

External Resources and AI Usage

- GitHub to collaborate: <https://github.com/oFeela/CSC3060-Project-4-Cache-Simulator>
- Bryan's chat: <https://chat.deepseek.com/share/zmj7wwelgtwgy9ru2>
- Geoffrey's chat: <https://chat.deepseek.com/share/8shf42y1al1mkkn5c>

Note

Please note that our trace (the one generated by the Python script) is `my_trace.txt` . But it was in UTF-16 encoding, so it didn't work during testing. We converted it to UTF-8 (the file `my_trace_utf8.txt`), and used this one instead for Task 3.